HW 4.3: Reproducible Data Analysis

Jifu Tang

2026-04-14

1 Busiest Airports Analysis

1.1 Airport Table

1.2 Airport Plot

1.3 Airport Narrative

Table Table 1 and Figure Figure 1 work together to show how passenger traffic changed for six major airports between 2020 and 2025. The line plot makes the overall pattern easy to see, while the table gives the exact values for each airport-year combination. Across both visualizations, the most noticeable feature is the general increase in passenger traffic after 2020. This pattern suggests a broad recovery in air travel over time.

Atlanta (ATL) remains the busiest airport in every year shown, and its line stays above the others throughout the figure. Dubai (DXB), Tokyo Haneda (HND), and Heathrow (LHR) also show strong growth after 2021. Frankfurt (FRA) increases too, but its recovery is slower and its passenger totals remain lower than those of ATL, DXB, HND, and LHR. Beijing Daxing (PKX) appears later and has missing values in several years, but the available values still show growth in the later part of the series.

Code	Airport	2020	2021	2022	2023	2024	2025
ATL	Hartsfield-Jackson Atlanta International Airport	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
DXB	Dubai International Airport	25,836,771	29,110,609	66,069,981	86,994,365	92,331,506	95,200,000
FRA	Frankfurt Airport	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
HND	Tokyo Haneda Airport	31,055,210	25,876,429	50,290,705	78,719,302	85,900,167	91,430,000
LHR	Heathrow Airport	22,111,469	NA	61,614,508	79,183,364	83,884,572	84,463,000
PKX	Beijing Daxing International Airport	NA	25,051,012	NA	NA	49,441,029	53,618,949

Table 1: Passenger traffic for six major airports from 2020 through 2025.

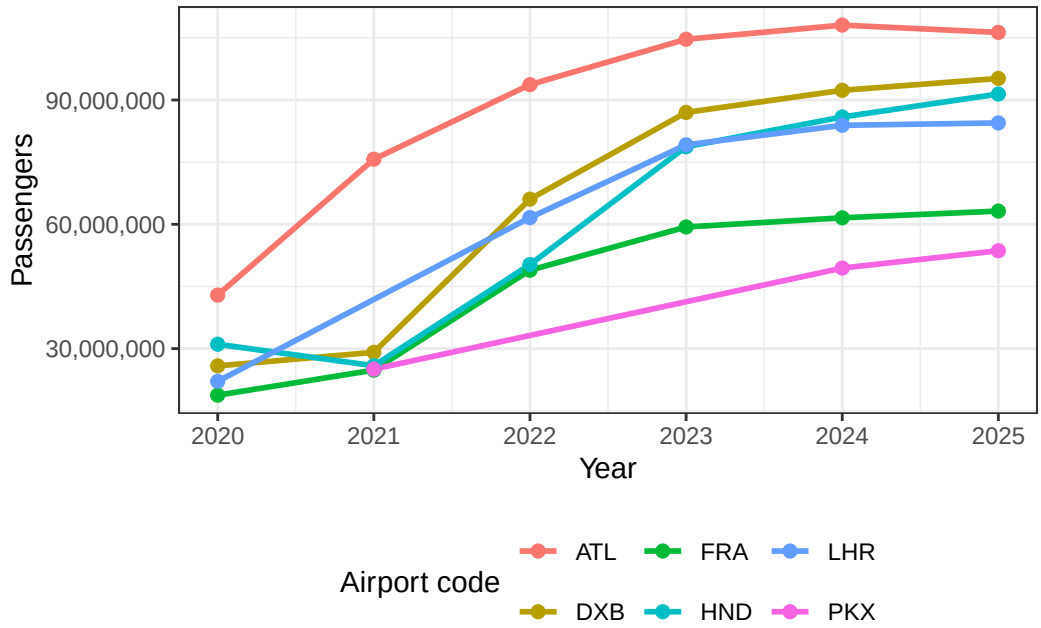


Figure 1: Passenger traffic for six major airports from 2020 through 2025.

The two visualizations also help the reader compare shape as well as size. For example, ATL rises quickly and then levels off slightly by 2025, while HND and DXB continue to climb in the later years. Because the table and plot support one another, they help build a clearer understanding of both the exact numbers and the larger travel trends across these airports.

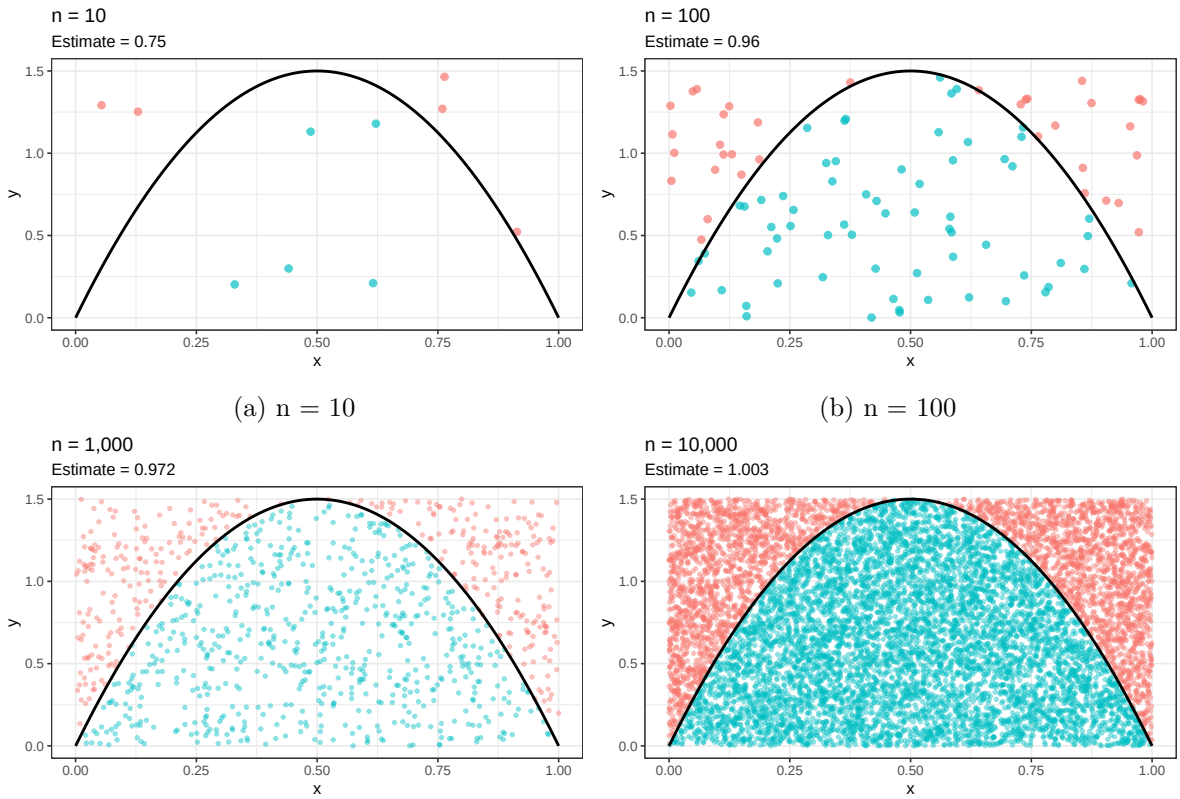
2 Monte Carlo Numerical Integration

2.1 Monte Carlo Process

Monte Carlo numerical integration uses random points to estimate area. In this assignment, the function is the beta density with parameters shape1 = 2 and shape2 = 2, the x-bounds are 0 to 1, and the y-bounds are 0 to 1.5. I first generate random (x, y) points inside that rectangle. Then I flag each point as either above the curve or on or below the curve. The proportion of points on or below the curve estimates the fraction of the rectangle that lies under the density, so multiplying that proportion by the rectangle area gives an estimate of the integral.

Because the rectangle has width 1 and height 1.5, its area is 1.5. As the number of simulated points increases, the estimate should become more stable because the random sample fills the rectangle more evenly.

2.2 Monte Carlo Plot



(a) n = 10

(b) n = 100

(c) n = 1,000

(d) n = 10,000

Figure 2: Monte Carlo integration for the beta density with parameters shape1 = 2 and shape2 = 2. In each panel, coral points fall above the curve and teal points fall on or below the curve.

2.3 Monte Carlo Narrative

Figure Figure 2 shows the same Monte Carlo process at four different resolutions. In the first panel, with only 10 points, the estimate changes a lot because a small number of random points can miss important parts of the curve. As the number of points increases to 100, 1,000, and 10,000, the rectangle becomes more fully sampled and the estimates become more stable. The later panels visually support this because the points fill the space much more evenly and the boundary of the curve is represented more clearly by the random sample.

I think the exact value of this integral is 1. The assigned function is a beta probability density function on the interval from 0 to 1, and a density function should have total area 1 over its

support. The small multiple supports this reasoning because the estimates move toward 1 as the number of points increases. The largest simulation is not exactly 1, but it is very close, which is what we would expect from a Monte Carlo estimate rather than an exact symbolic calculation.

The figure also helps explain why this happens. In the larger panels, the proportion of teal points under the curve becomes more consistent from region to region, so the estimated area settles near the true value. This makes the visual evidence compatible with the mathematical fact that the integral of the assigned beta density should equal 1.

3 Planning and Prompting GenAI Tools

3.1 Calcium Context

The calcium data come from a longitudinal study of 31 women followed across four years. The first four columns belong to the null treatment group and the second four columns belong to the diet and exercise treatment group. The file is not tidy because the repeated measurements are stored across columns rather than in a single measurement column with separate variables for group and year.

3.2 Plan

3.2.1 Goal

My goal is to tidy the calcium data and create a visualization that helps a reader understand how ulnar calcium measurements change over time and whether those patterns differ between the null treatment group and the diet and exercise treatment group.

3.2.2 Needs

To do this well, I need the original `calcium.csv` file, a clear way to identify which columns belong to each treatment group, a time variable that distinguishes the initial measurement from Years 1 through 3, and a plot that respects the longitudinal structure of the study. I also need to preserve the idea that each row corresponds to a woman rather than treating the original columns as unrelated variables.

3.2.3 Steps

1. Read the original `calcium.csv` file without changing the file itself.
2. Separate the first four columns from the second four columns so the two treatment groups are identified correctly.
3. Rename the repeated-measurement columns so the time points are meaningful and readable.
4. Add an identifier for each woman inside each treatment group.
5. Reshape the data from wide format to long format so there is one calcium measurement per row.
6. Check for missing values and keep them from distorting summaries or plots.
7. Create a visualization that shows both treatment group and time, rather than only plotting column averages.
8. Compare the resulting visualization with a generic prompt response to see how planning changes the quality of the result.

3.3 Plan-Informed Response

When I gave ChatGPT both the full plan and the original calcium file, the response moved in a more context-aware direction. Instead of treating the columns as unrelated variables, the tool recognized that the data should first be tidied and that the repeated measurements represented the same women across time. It recommended reshaping the file into a long format with variables for treatment group, participant, year, and calcium value. It also recommended using a plot that preserved the longitudinal structure, such as a faceted line plot or another time-based visualization that compares the two treatment groups over time.

One useful part of the plan-informed response was that it aligned the structure of the data with the structure of the scientific question. The point was not simply to compare column means. The point was to compare change across time and treatment groups. That is a better match between the context and the visualization task.

3.3.1 Plan-Informed Response Text

> To match your plan and the study design, I would first separate the first four columns from the second four columns so the treatment groups are identified correctly. Then I would rename the repeated-measurement columns to meaningful time labels such as Initial, Year 1, Year 2, and Year 3. After that, I would add a participant identifier within each treatment group and reshape the file from wide format to long format so that each row contains one calcium measurement along with the participant, treatment group, and year.

> For the visualization, I would avoid a simple plot of column averages because that would ignore both the treatment structure and the longitudinal structure. A better choice would be a time-based plot that preserves repeated measurements, such as a faceted line plot with one panel for each treatment group and an overlaid summary line showing the group mean across time. That approach would let the reader compare overall change across years while still respecting the original cases in the data.

3.4 Generic Response

The generic response did not begin by tidying the data or identifying the treatment groups. Instead, it immediately created the plot shown in Figure Figure 3. That figure treats each original column as a separate category and plots only the average value for each column.

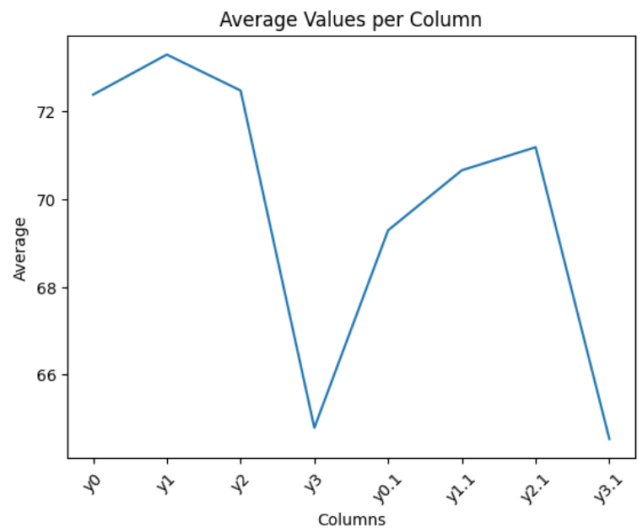


Figure 3: Generic ChatGPT response using a prompt with no plan. The figure plots the average of each original column rather than representing treatment group and time in a tidy way.

The accompanying text response was also very general:

> I created a visualization showing the average value of each column in your dataset.
>
> What this shows:
>
> Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.).
>
> You can quickly compare which variables tend to have higher or lower values overall.
>
> For example, some columns cluster around ~70–73, while others drop closer to ~65.

This response is not completely wrong, but it is shallow. It ignores the structure of the study, does not separate the two treatment groups in a meaningful way, and does not help the reader understand longitudinal change very well.

3.5 My Own Visualization

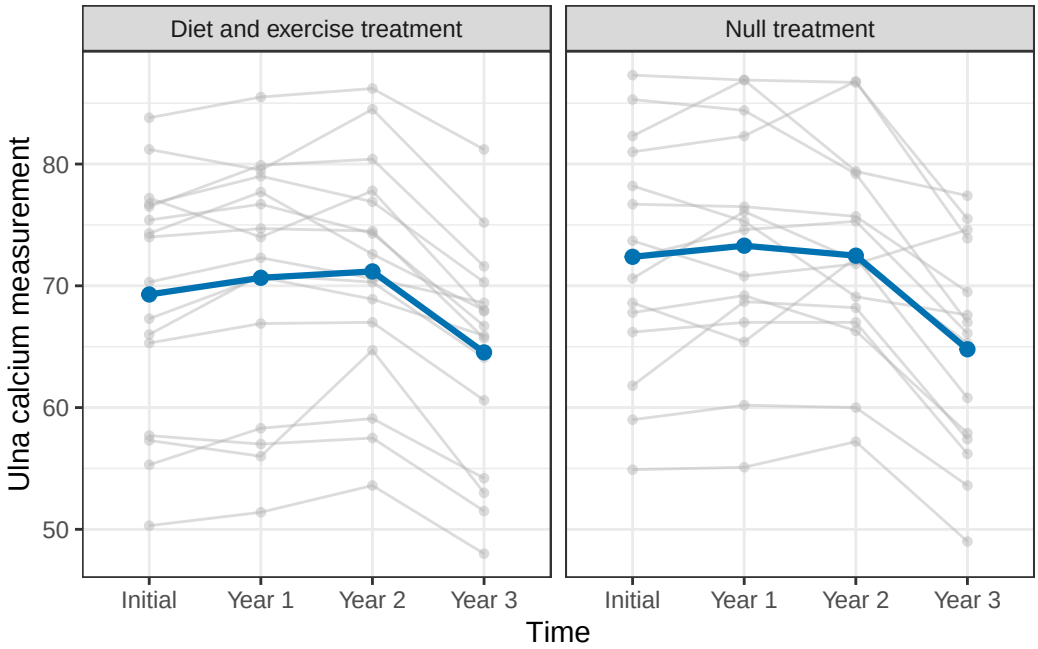


Figure 4: A plan-informed visualization of the calcium data. Individual women are shown with faint lines, and the thicker line in each panel shows the mean calcium level for that treatment group across time.

Figure Figure 4 is my own reproducible visualization after tidying the data. It keeps the two treatment groups separate and shows time in the correct order. The faint gray lines preserve the individual longitudinal records, while the thicker blue mean line helps summarize the overall pattern in each treatment group.

3.6 Comparison

The differences between the three artifacts are meaningful. The generic response in Figure Figure 3 treats the original columns as the main story. That approach loses the idea of case because the women disappear, and it also loses the study structure because treatment group and time are mixed together as column names. In contrast, the plan-informed response recognized that the data first needed to be tidied so that the cases, treatment groups, and

repeated measurements were represented correctly. My own visualization in Figure Figure 4 follows that plan and therefore better matches the scientific question.

The visual differences matter because they change what the reader can learn. The generic plot only shows that some column means are higher than others. It does not clearly communicate that both groups have repeated measurements over time, nor does it help the reader compare trajectories between groups. My own plot shows that the two groups have fairly similar average calcium levels through Year 2 and that both groups decline by Year 3. It also shows the amount of person-to-person variation within each group, which the generic plot hides.

This comparison taught me that better prompting is not just about getting a longer answer. A detailed plan changes the tool's understanding of the situation, so the response is more likely to respect the structure of the data and the actual context of the problem. It also taught me that genAI still needs human judgment. Even when the plan-informed response was more useful than the generic one, I still needed to decide how to tidy the file, how to encode the time variable, and what plot would best support the comparison.

4 Reflection

This course has helped me move from simply running code to thinking more carefully about how data, code, and communication fit together. One thing I learned is that a good visualization is not just something that looks nice. It needs to match the structure of the data and the question being asked. The busiest airports section reminded me that tables and plots do different jobs: the table supports exact comparison, while the plot makes trends easier to see. I also learned that a narrative should explain what visual features matter instead of only repeating what is already visible.

Another thing I learned is how reproducibility changes the way I think about analysis. In earlier work, I mainly focused on getting an answer. In this assignment, I had to organize the work into a document that someone else could read and follow. That pushed me to think about file structure, section headings, captions, and code organization instead of treating those details as unimportant.

The Monte Carlo section also helped me better understand the relationship between simulation and mathematical reasoning. Before, I could follow the steps mechanically, but now I better understand why larger sample sizes produce more stable estimates and how the visual pattern in the small multiple supports the exact value of the integral. The Planning and Prompting section taught me something similar about genAI: a tool can be helpful, but the quality of the result depends strongly on whether the user understands the data and asks a question that respects the context.

5 GenAI Usage Appendix

5.1 Usage 1: Busiest Airports Narrative

Tool: ChatGPT (GPT-5.4 Thinking)
Date: April 14, 2026

Exact prompt(s):

- "Use my existing busiest airports table, plot, and homework analysis to help me rewrite this section as a short report paragraph for a Quarto document. Keep the core ideas the same, but make the writing clearer and more connected to both the table and the plot."

Exact response text generated by the tool:

> Table Table 1 and Figure Figure 1 work together to show how passenger traffic changed for six major airports between 2020 and 2025. The line plot makes the overall pattern easy to see, while the table gives the exact values for each airport-year combination. Across both visualizations, the most noticeable feature is the general increase in passenger traffic after 2020. This pattern suggests a broad recovery in air travel over time.
>
> Atlanta (ATL) remains the busiest airport in every year shown, and its line stays above the others throughout the figure. Dubai (DXB), Tokyo Haneda (HND), and Heathrow (LHR) also show strong growth after 2021. Frankfurt (FRA) increases too, but its recovery is slower and its passenger totals remain lower than those of ATL, DXB, HND, and LHR. Beijing Daxing (PKX) appears later and has missing values in several years, but the available values still show growth in the later part of the series.
>
> The two visualizations also help the reader compare shape as well as size. For example, ATL rises quickly and then levels off slightly by 2025, while HND and DXB continue to climb in the later years. Because the table and plot support one another, they help build a clearer understanding of both the exact numbers and the larger travel trends across these airports.

5.2 Usage 2: Monte Carlo Narrative

Tool: ChatGPT (GPT-5.4 Thinking)
Date: April 14, 2026

Exact prompt(s):

- "Use my existing Monte Carlo small multiple and homework explanation to help me rewrite this section for a Quarto report. Keep the same main idea that the integral is 1 and explain what in the visualization supports that reasoning."

Exact response text generated by the tool:

> Figure Figure 2 shows the same Monte Carlo process at four different resolutions. In the first panel, with only 10 points, the estimate changes a lot because a small number of random points can miss important parts of the curve. As the number of points increases to 100, 1,000, and 10,000, the rectangle becomes more fully sampled and the estimates become more stable. The later panels visually support this because the points fill the space much more evenly and the boundary of the curve is represented more clearly by the random sample.
>
> I think the exact value of this integral is 1. The assigned function is a beta probability density function on the interval from 0 to 1, and a density function should have total area 1 over its support. The small multiple supports this reasoning because the estimates move toward 1 as the number of points increases. The largest simulation is not exactly 1, but it is very close, which is what we would expect from a Monte Carlo estimate rather than an exact symbolic calculation.
>
> The figure also helps explain why this happens. In the larger panels, the proportion of teal points under the curve becomes more consistent from region to region, so the estimated area settles near the true value. This makes the visual evidence compatible with the mathematical fact that the integral of the assigned beta density should equal 1.

5.3 Usage 3: Plan-Informed Prompting with Calcium Data

Tool: ChatGPT (GPT-5.4 Thinking)
Date: April 14, 2026

Exact prompt(s):

- "I have a calcium.csv file from a longitudinal study of 31 women followed over four years. The first four columns belong to the null treatment group and the second four columns belong to the diet and exercise treatment group. The file is not tidy. I want to tidy the data and create a plot that helps a reader understand how ulnar calcium measurements change over time and whether the patterns differ between treatment groups. My plan is to identify the two groups, rename the measurement columns so the time points are clear, add an identifier for each woman, reshape the file from wide to long format, check missing values, and then create a visualization that respects both treatment group and time. Based on that plan, what wrangling and visualization approach would you recommend?"

Exact response text generated by the tool:

> To match your plan and the study design, I would first separate the first four columns from the second four columns so the treatment groups are identified correctly. Then I would rename the repeated-measurement columns to meaningful time labels such as Initial, Year 1, Year 2, and Year 3. After that, I would add a participant identifier within each treatment group and reshape the file from wide format to long format so that each row contains one calcium measurement along with the participant, treatment group, and year.
>
> For the visualization, I would avoid a simple plot of column averages because that would ignore both the treatment structure and the longitudinal structure. A better choice would be a time-based plot that preserves repeated measurements, such as a faceted line plot with one panel for each treatment group and an overlaid summary line showing the group mean across time. That approach would let the reader compare overall change across years while still respecting the original cases in the data.

5.4 Usage 4: Generic Response Materials Supplied in the Assignment

Tool: OpenAI ChatGPT (GPT-5.3)
Date: April 14, 2026
Source: Generic response materials provided in the assignment

Exact prompt(s):

- "Create a data visualization for these data. Calcium.csv"
- "What is your version information?"

Exact response text generated by the tool:

> I created a visualization showing the average value of each column in your dataset.
>
> What this shows:
>
> Each point represents the mean (average) of one column (y0, y1, y2, y3, etc.).
>
> You can quickly compare which variables tend to have higher or lower values overall.
>
> For example, some columns cluster around ~70–73, while others drop closer to ~65.
>
> If you want something more insightful, I can also create:
>
> Boxplots (to show spread and outliers)
>
> Time series plots (if rows represent time/order)
>
> Correlation heatmaps

Side-by-side comparisons (e.g., y0 vs y0.1)

Just tell me

6 Code Appendix

```r
# ------------------------------------------------------------
# Setup
# ------------------------------------------------------------

library(tidyverse)
library(knitr)
library(kableExtra)
library(scales)

# ------------------------------------------------------------
# Busiest Airports Data
# ------------------------------------------------------------

busiest_airports_tidy <- tibble(
  airport_code = c(
    "ATL", "ATL", "ATL", "ATL", "ATL", "ATL",
    "DXB", "DXB", "DXB", "DXB", "DXB", "DXB",
    "FRA", "FRA", "FRA", "FRA", "FRA", "FRA",
    "HND", "HND", "HND", "HND", "HND", "HND",
    "LHR", "LHR", "LHR", "LHR", "LHR", "LHR",
    "PKX", "PKX", "PKX", "PKX", "PKX", "PKX"
  ),
  airport_name = c(
    rep("Hartsfield-Jackson Atlanta International Airport", 6),
    rep("Dubai International Airport", 6),
    rep("Frankfurt Airport", 6),
    rep("Tokyo Haneda Airport", 6),
    rep("Heathrow Airport", 6),
    rep("Beijing Daxing International Airport", 6)
  ),
  year = rep(2020:2025, 6),
  passengers = c(
    42918685, 75704760, 93699630, 104653451, 108067766, 106302208,
    25836771, 29110609, 66069981, 86994365, 92331506, 95200000,
    18768601, 24812849, 48918482, 59355389, 61564957, 63189666,
    31055210, 25876429, 50290705, 78719302, 85900167, 91430000,
    22111469, NA, 61614508, 79183364, 83884572, 84463000,
    NA, 25051012, NA, NA, 49441029, 53618949
  )
```

```r
)

# ------------------------------------------------------------
# Busiest Airports Table
# ------------------------------------------------------------

airport_table <- busiest_airports_tidy |>
  mutate(
    passengers = if_else(is.na(passengers), NA_character_, comma(passengers))
  ) |>
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) |>
  arrange(airport_code)

airport_table |>
  kbl(
    col.names = c("Code", "Airport", "2020", "2021", "2022", "2023", "2024", "2025"),
    booktabs = TRUE
  ) |>
  kable_styling(
    latex_options = c("hold_position", "scale_down"),
    full_width = FALSE
  )

# ------------------------------------------------------------
# Busiest Airports Plot
# ------------------------------------------------------------

busiest_airports_tidy |>
  filter(!is.na(passengers)) |>
  ggplot(
    aes(
      x = year,
      y = passengers,
      color = airport_code,
      group = airport_code
    )
  ) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
```

```r
  scale_x_continuous(breaks = 2020:2025) +
  scale_y_continuous(labels = comma) +
  labs(
    x = "Year",
    y = "Passengers",
    color = "Airport code"
  ) +
  theme_bw() +
  theme(legend.position = "bottom")

# ------------------------------------------------------------
# Monte Carlo Functions
# ------------------------------------------------------------

mc_sim <- function(n, x_min, x_max, y_min, y_max) {
  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)

  tibble(
    x = x_vals,
    y = y_vals
  )
}

make_mc_plot <- function(n_value) {
  sim_data <- mc_sim(
    n = n_value,
    x_min = 0,
    x_max = 1,
    y_min = 0,
    y_max = 1.5
  ) |>
    mutate(
      f_x = dbeta(x, shape1 = 2, shape2 = 2),
      flag = if_else(y <= f_x, "on/below", "above")
    )

  p_hat <- mean(sim_data$flag == "on/below")
  estimate <- 1.5 * p_hat

  point_alpha <- if_else(n_value >= 1000, 0.45, 0.70)
  point_size <- if_else(n_value >= 1000, 1.0, 2.0)
```

```r
  ggplot(sim_data, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = point_alpha, size = point_size) +
    stat_function(
      fun = dbeta,
      args = list(shape1 = 2, shape2 = 2),
      xlim = c(0, 1),
      linewidth = 0.9,
      color = "black"
    ) +
    scale_color_manual(
      values = c("above" = "#F8766D", "on/below" = "#00BFC4")
    ) +
    coord_cartesian(xlim = c(0, 1), ylim = c(0, 1.5)) +
    labs(
      title = paste("n =", comma(n_value)),
      subtitle = paste("Estimate =", round(estimate, 4)),
      x = "x",
      y = "y"
    ) +
    theme_bw() +
    theme(legend.position = "none")
}

# ------------------------------------------------------------
# Monte Carlo Small Multiple Objects
# ------------------------------------------------------------

set.seed(184)
plot10 <- make_mc_plot(10)
plot100 <- make_mc_plot(100)
plot1000 <- make_mc_plot(1000)
plot10000 <- make_mc_plot(10000)

plot10
plot100
plot1000
plot10000

# ------------------------------------------------------------
# Calcium Data Wrangling
# ------------------------------------------------------------
```

```r
calcium_raw <- read_csv("calcium.csv", show_col_types = FALSE)

null_group <- calcium_raw |>
  select(y0, y1, y2, y3) |>
  rename(
    Initial = y0,
    `Year 1` = y1,
    `Year 2` = y2,
    `Year 3` = y3
  ) |>
  mutate(
    group = "Null treatment",
    participant = row_number()
  )

diet_group <- calcium_raw |>
  select(`y0.1`, `y1.1`, `y2.1`, `y3.1`) |>
  rename(
    Initial = `y0.1`,
    `Year 1` = `y1.1`,
    `Year 2` = `y2.1`,
    `Year 3` = `y3.1`
  ) |>
  mutate(
    group = "Diet and exercise treatment",
    participant = row_number()
  )

calcium_tidy <- bind_rows(null_group, diet_group) |>
  pivot_longer(
    cols = c(Initial, `Year 1`, `Year 2`, `Year 3`),
    names_to = "year",
    values_to = "calcium"
  ) |>
  drop_na(calcium) |>
  mutate(
    year = factor(year, levels = c("Initial", "Year 1", "Year 2", "Year 3"))
  )

# ------------------------------------------------------------
# Calcium Plot
# ------------------------------------------------------------
```

```r
ggplot(
  calcium_tidy,
  aes(x = year, y = calcium, group = participant)
) +
  geom_line(color = "gray70", alpha = 0.45) +
  geom_point(color = "gray70", alpha = 0.45, size = 1.2) +
  stat_summary(
    aes(group = 1),
    fun = mean,
    geom = "line",
    linewidth = 1.1,
    color = "#0072B2"
  ) +
  stat_summary(
    aes(group = 1),
    fun = mean,
    geom = "point",
    size = 2.2,
    color = "#0072B2"
  ) +
  facet_wrap(~ group) +
  labs(
    x = "Time",
    y = "Ulna calcium measurement"
  ) +
  theme_bw()
```